

ACTIVITY NO. 6	
SEARCHING TECHNIQUES	
Course Code: CPE010	Program: Computer Engineering
Course Title: Data Structures and Algorithms	Date Performed: October 14, 2024
Section: CPE21S4	Date Submitted: October 16, 2024
Name: Gob, Mark Jeonel Kenn E.	Instructor: Ma'am Maria Rizette Sayo

6. Output

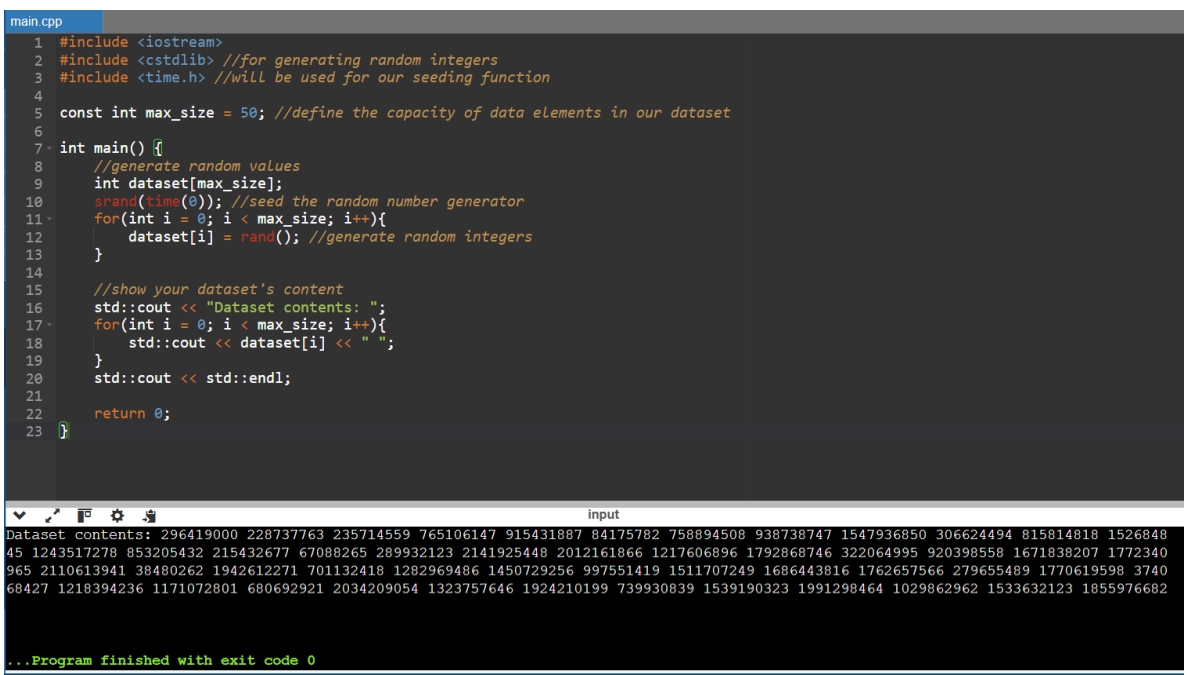
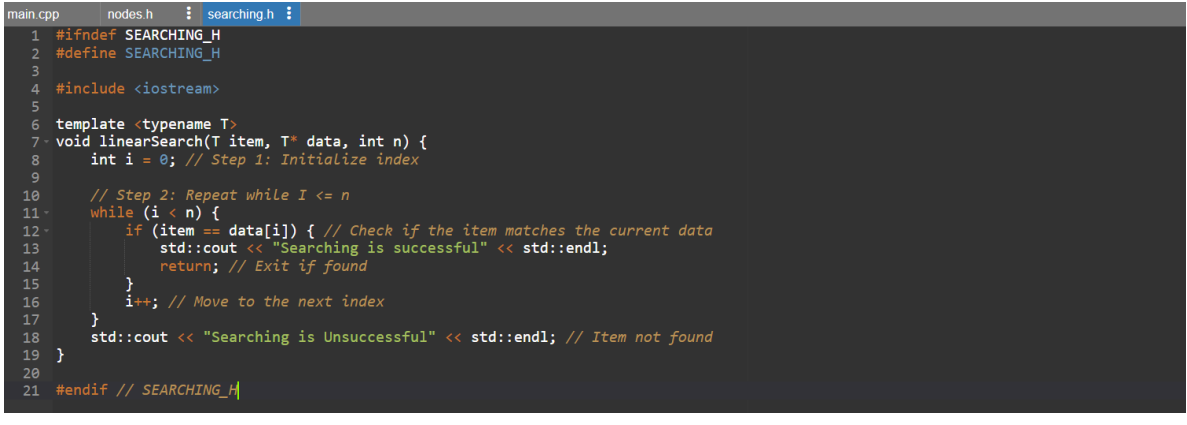
Screenshot	 <pre>main.cpp 1 #include <iostream> 2 #include <cstdlib> //for generating random integers 3 #include <time.h> //will be used for our seeding function 4 5 const int max_size = 50; //define the capacity of data elements in our dataset 6 7 int main() { 8 //generate random values 9 int dataset[max_size]; 10 srand(time(0)); //seed the random number generator 11 for(int i = 0; i < max_size; i++){ 12 dataset[i] = rand(); //generate random integers 13 } 14 15 //show your dataset's content 16 std::cout << "Dataset contents: "; 17 for(int i = 0; i < max_size; i++){ 18 std::cout << dataset[i] << " "; 19 } 20 std::cout << std::endl; 21 22 return 0; 23 }</pre> input Dataset contents: 296419000 228737763 235714559 765106147 915431887 84175782 758894508 938738747 1547936850 306624494 815814818 1526848 45 1243517278 853205432 215432677 67088265 289932123 2141925448 2012161866 1217606896 1792868746 322064995 920398558 1671838207 1772340 965 2110613941 38480262 1942612271 701132418 1282969486 1450729256 997551419 1511707249 1686443816 1762657566 279655489 1770619598 3740 68427 1218394236 1171072801 680692921 2034209054 1323757646 1924210199 739930839 1539190323 1991298464 1029862962 1533632123 1855976682 ...Program finished with exit code 0
Observations	I observe that the code generates an array of 50 random integers and displays them on the output console. The srand function is used to seed the random number generator with the current time, ensuring that a different sequence of random numbers is generated each time the program is run.

Table 6-1. Data Generated and Observations.

Code	 <pre>main.cpp nodes.h : searching.h : 1 #ifndef SEARCHING_H 2 #define SEARCHING_H 3 4 #include <iostream> 5 6 template <typename T> 7 void linearSearch(T item, T* data, int n) { 8 int i = 0; // Step 1: Initialize index 9 10 // Step 2: Repeat while I <= n 11 while (i < n) { 12 if (item == data[i]) { // Check if the item matches the current data 13 std::cout << "Searching is successful" << std::endl; 14 return; // Exit if found 15 } 16 i++; // Move to the next index 17 } 18 std::cout << "Searching is Unsuccessful" << std::endl; // Item not found 19 } 20 21 #endif // SEARCHING_H</pre>
------	--

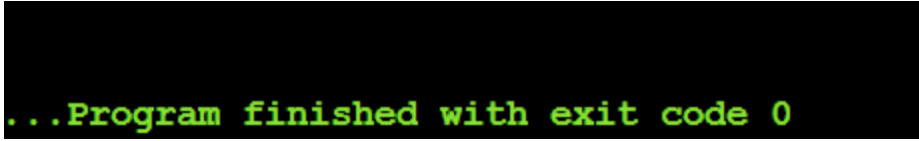
Output	
Observations	I observe that the searching.h code provides a correct implementation of the linear search algorithm, using templates to make it flexible and reusable for various data types. However, the function lacks error handling mechanisms, such as input validation or try-catch blocks, which could improve it. Additionally, the function's output is limited to printing success or failure messages, which might not be suitable for all use cases.

Table 6-2a. Linear Search for Arrays

Code	<pre> 1 #include <iostream> 2 3 // Define the Node structure 4 template <typename T> 5 struct Node { 6 T data; // Data stored in the node 7 Node* next; // Pointer to the next node 8 }; 9 10 // Function to create a new node 11 template <typename T> 12 Node<T>* new_node(T value) { 13 Node<T>* newNode = new Node<T>(); // Allocate memory for a new node 14 newNode->data = value; // Set the data 15 newNode->next = nullptr; // Initialize next pointer to null 16 return newNode; // Return the newly created node 17 } 18 19 // Linear search function for Linked List 20 template <typename T> 21 void linearLS(Node<T>* head, T dataFind) { 22 Node<T>* current = head; // Initialize current node to head 23 int position = 1; // Initialize position counter 24 25 while (current != nullptr) { // Traverse the Linked List 26 if (current->data == dataFind) { // Check if the current node's data matches the search value 27 std::cout << "Element " << dataFind << " found at position " << position << std::endl; 28 return; // Exit if found 29 } 30 current = current->next; // Move to the next node 31 position++; // Increment position counter 32 } 33 34 std::cout << "Element " << dataFind << " not found in the linked list." << std::endl; // Item not found 35 } 36 37 int main() { 38 // Create Linked List for the name "Kenn" 39 Node<char>* name1 = new_node('K'); 40 Node<char>* name2 = new_node('e'); 41 Node<char>* name3 = new_node('n'); 42 Node<char>* name4 = new_node('n'); 43 44 // Link each node to each other 45 name1->next = name2; 46 name2->next = name3; 47 name3->next = name4; 48 name4->next = nullptr; 49 50 // Perform Linear search 51 linearLS(name1, 'e'); // Searching for 'e' 52 53 // Clean up memory (optional) 54 Node<char>* current = name1; 55 while (current != nullptr) { 56 Node<char>* nextNode = current->next; 57 delete current; // Deallocate memory for the current node 58 current = nextNode; // Move to the next node 59 } 60 61 return 0; 62 } </pre>
------	--

Output	<pre> Element e found at position 2 ...Program finished with exit code 0 Press ENTER to exit console. </pre>
Observations	I observe that my modified linear search algorithm in C++ to indicate the position of the element found in a linked list is a good approach to searching for a specific element within a linked list. Moreover, the key observation here is that the algorithm introduces a position counter to keep track of the current position during the traversal, allowing it to accurately report the position of the found element.

Table 6-2b. Linear Search for Linked List

Code	<pre> 1 #include <iostream> 2 3 // Define the structure for a Linked List node 4 struct Node { 5 int data; 6 Node* next; 7 }; 8 9 // Function to find the middle element of a Linked List 10 Node* findMiddle(Node* start, Node* last) { 11 if (start == NULL) { 12 return NULL; 13 } 14 15 if (start == last) { 16 return start; 17 } 18 19 Node* slow = start; 20 Node* fast = start->next; 21 22 while (fast != last) { 23 fast = fast->next; 24 slow = slow->next; 25 if (fast != last) { 26 fast = fast->next; 27 } 28 } 29 30 return slow; 31 } 32 33 // Function to perform binary search on a Linked List 34 bool binarySearch(Node* head, int value) { 35 Node* start = head; 36 Node* last = NULL; 37 int position = 1; 38 39 while (true) { 40 // Find the middle element 41 Node* mid = findMiddle(start, last); 42 43 // If the middle element is empty 44 if (mid == NULL) { 45 std::cout << "Value " << value << " not found in the linked list." << std::endl; 46 return false; 47 } 48 49 // If the value is present at the middle element 50 if (mid->data == value) { 51 std::cout << "Value " << value << " found at position " << position << std::endl; 52 return true; 53 } 54 55 // If the start and last node are overlapping 56 else if (start == last) { 57 break; 58 } 59 60 // If the value is less than the middle element 61 else if (mid->data < value) { 62 start = mid->next; 63 position++; 64 } 65 } </pre>
------	--

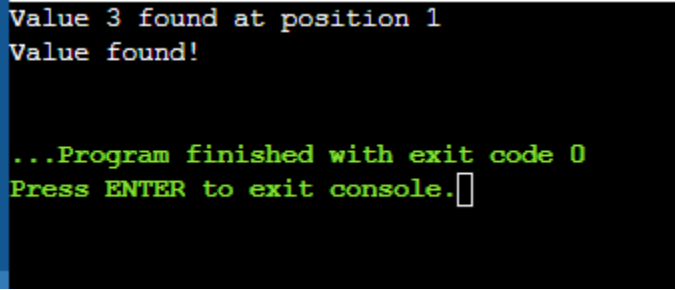
	<pre>66 // If the value is greater than the middle element 67 else { 68 last = mid; 69 } 70 } 71 72 // Value not present 73 std::cout << "Value " << value << " not found in the linked list." << std::endl; 74 return false; 75 } 76 77 int main() { 78 // Create a sample Linked List 79 Node* head = new Node(); 80 head->data = 1; 81 head->next = new Node(); 82 head->next->data = 2; 83 head->next->next = new Node(); 84 head->next->next->data = 3; 85 head->next->next->next = new Node(); 86 head->next->next->next->data = 4; 87 head->next->next->next->next = new Node(); 88 head->next->next->next->next->data = 5; 89 90 // Perform binary search 91 int value = 3; 92 if (binarySearch(head, value)) { 93 std::cout << "Value found!" << std::endl; 94 } else { 95 std::cout << "Value not found!" << std::endl; 96 } 97 98 return 0; 99 }</pre>
Output	
Observations	This code searches for a specific element within a linked list, which provides feedback to the user about the location of the searched element. My key observation here is this binary search algorithm leverages something like a divide-and-conquer method, repeatedly dividing the search space in half to pinpoint the target element, and incorporates a position counter to accurately report the element's position. By combining these features, the algorithm provides a clear output, indicating the value found and its position in the linked list.

Table 6-3a. Binary Search for Arrays

Code

```
1  #include <iostream>
2
3  using namespace std;
4
5  // Node structure for the Linked List
6  template <typename T>
7  struct Node {
8      T data;
9      Node<T>* next;
10 };
11
12 // Function to create a new node
13 template <typename T>
14 Node<T>* new_node(T data) {
15     Node<T>* newNode = new Node<T>;
16     newNode->data = data;
17     newNode->next = NULL;
18     return newNode;
19 }
20
21 // Function to find the middle node
22 template <typename T>
23 Node<T>* findMiddle(Node<T>* start, Node<T>* last) {
24     if (start == NULL) return NULL;
25
26     Node<T>* slow = start;
27     Node<T>* fast = start;
28
29     while (fast != last && fast->next != last) {
30         fast = fast->next->next;
31         slow = slow->next;
32     }
33
34     return slow;
35 }
36
37 // Function to perform binary search
38 template <typename T>
39 bool binarySearch(Node<T>* head, T value) {
40     Node<T>* start = head;
41     Node<T>* last = NULL;
42     int position = 1;
43
44     while (true) {
45         Node<T>* mid = findMiddle(start, last);
46
47         if (mid == NULL) {
48             cout << "Value " << value << " not found in the linked list." << endl;
49             return false;
50         }
51     }
```

```

50     }
51
52     if (mid->data == value) {
53         cout << "Value " << value << " found at position " << position << endl;
54         return true;
55     }
56
57     else if (start == last) {
58         break;
59     }
60
61     else if (mid->data < value) {
62         start = mid->next;
63         position++;
64     }
65
66     else {
67         last = mid;
68     }
69 }
70
71 cout << "Value " << value << " not found in the linked list." << endl;
72 return false;
73 }
74
75 int main() {
76     char choice = 'y';
77     int count = 1;
78     int newData;
79     Node<int>* temp, *head, *node;
80
81     while (choice == 'y') {
82         cout << "Enter data: ";
83         cin >> newData;
84
85         if (count == 1) {
86             head = new_node(newData);
87             cout << "Successfully added " << head->data << " to the list.\n";
88             count++;
89         } else if (count == 2) {
90             node = new_node(newData);
91             head->next = node;
92             node->next = NULL;
93             cout << "Successfully added " << node->data << " to the list.\n";
94             count++;
95         } else {
96             temp = head;
97             while (true) {
98                 if (temp->next == NULL) break;
99                 temp = temp->next;
100             }
101             node = new_node(newData);
102             temp->next = node;

```

```

103         cout << "Successfully added " << node->data << " to the list.\n";
104         count++;
105     }
106
107     cout << "Continue? (y/n)";
108     cin >> choice;
109     if (choice == 'n') break;
110 }
111
112 // Display the Linked List
113 Node<int>* currNode;
114 currNode = head;
115 cout << "Linked list: ";
116 while (currNode != NULL) {
117     cout << currNode->data << " ";
118     currNode = currNode->next;
119 }
120 cout << endl;
121
122 // Test the binary search function
123 int searchValue;
124 cout << "Enter a value to search: ";
125 cin >> searchValue;
126 binarySearch(head, searchValue);
127
128 return 0;
129 }

```

Output

Output for the 3 cases:

```

Enter data: 10
Successfully added 10 to the list.
Continue? (y/n)y
Enter data: 20
Successfully added 20 to the list.
Continue? (y/n)y
Enter data: 30
Successfully added 30 to the list.
Continue? (y/n)n
Linked list: 10 20 30
Enter a value to search: 20
Value 20 found at position 2

...Program finished with exit code 0
Press ENTER to exit console.

```

```

Enter data: 10
Successfully added 10 to the list.
Continue? (y/n)y
Enter data: 20
Successfully added 20 to the list.
Continue? (y/n)y
Enter data: 30
Successfully added 30 to the list.
Continue? (y/n)n
Linked list: 10 20 30
Enter a value to search: 10
Value 10 found at position 1

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

```

Enter data: 10
Successfully added 10 to the list.
Continue? (y/n)y
Enter data: 20
Successfully added 20 to the list.
Continue? (y/n)n
Linked list: 10 20
Enter a value to search: 30
Value 30 not found in the linked list.

...Program finished with exit code 0
Press ENTER to exit console.

```

Observations

Case 1: Value found in the middle of the linked list
The binary search efficiently locates the value 20 at position 2, demonstrating its effectiveness in a sorted linked list.

Case 2: Value found at the beginning of the linked list
The algorithm successfully identifies the value 10 at position 1, showcasing its capability to handle edge cases.

Case 3: Value not found in the linked list

The search correctly concludes that the value is not present in the linked list, providing clear user feedback.

Table 6-3b. Binary Search for Linked List

7. Supplementary Activity

ILO B: Solve different problems utilizing appropriate searching techniques in C++

For each provided problem, give a screenshot of your code, the output console, and your answers to the questions.

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'? - There would be 2.

```
1 #include <iostream>
2
3 struct Node {
4     int data;
5     Node* next;
6 };
7
8 // Function to create a new node
9 Node* createNode(int data) {
10     Node* newNode = new Node();
11     if (newNode == NULL) {
12         std::cout << "Memory error\n";
13         return NULL;
14     }
15     newNode->data = data;
16     newNode->next = NULL;
17     return newNode;
18 }
19
20 // Function to insert a node at the end of the Linked List
21 void insertNode(Node* head, int data) {
22     Node* newNode = createNode(data);
23     if (head == NULL) {
24         head = newNode;
25         return;
26     }
27     Node* lastNode = head;
28     while (lastNode->next != NULL) {
29         lastNode = lastNode->next;
30     }
31     lastNode->next = newNode;
32 }
33
34 // Function to perform sequential search on linked list
35 int sequentialSearchLinkedList(Node* head, int key) {
36     int comparisons = 0;
37     while (head != nullptr) {
38         comparisons++;
39         if (head->data == key) {
40             std::cout << "Key found! Comparisons: " << comparisons << std::endl;
41             return comparisons;
42         }
43         head = head->next;
44     }
45     std::cout << "Key not found. Comparisons: " << comparisons << std::endl;
46     return comparisons;
47 }
48
49 // Function to perform sequential search on array
50 int sequentialSearchArray(int arr[], int size, int key) {
51     int comparisons = 0;
52     for (int i = 0; i < size; i++) {
53         comparisons++;
54         if (arr[i] == key) {
55             std::cout << "Key found! Comparisons: " << comparisons << std::endl;
56             return comparisons;
57         }
58     }
59     std::cout << "Key not found. Comparisons: " << comparisons << std::endl;
60     return comparisons;
61 }
62
63 int main() {
64     // Linked List approach
65     Node* head = NULL;
66     int linkedListData[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
67     int linkedListSize = sizeof(linkedListData) / sizeof(linkedListData[0]);
68     for (int i = 0; i < linkedListSize; i++) {
69         insertNode(head, linkedListData[i]);
70     }
71     int comparisonsLinkedList = sequentialSearchLinkedList(head, 18);
72
73     // Array approach
74     int arr[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
75     int size = sizeof(arr) / sizeof(arr[0]);
76     int comparisonsArray = sequentialSearchArray(arr, size, 18);
77     return 0;
78 }
```

Output:

```
Key found! Comparisons: 2
Key found! Comparisons: 2

...Program finished with exit code 0
Press ENTER to exit console.
```

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.


```

1 #include <iostream>
2
3 struct Node {
4     int data;
5     Node* next;
6 };
7
8 // Function to create a new node
9 Node* createNode(int data) {
10     Node* newNode = new Node();
11     if (newNode == NULL) {
12         std::cout << "Memory error\n";
13         return NULL;
14     }
15     newNode->data = data;
16     newNode->next = NULL;
17     return newNode;
18 }
19
20 // Function to insert a node at the end of the Linked List
21 void insertNode(Node*& head, int data) {
22     Node* newNode = createNode(data);
23     if (head == NULL) {
24         head = newNode;
25         return;
26     }
27     Node* lastNode = head;
28     while (lastNode->next != NULL) {
29         lastNode = lastNode->next;
30     }
31     lastNode->next = newNode;
32 }
33
34 // Function to count occurrences of 'key' in linked list
35 int countOccurrencesLinkedList(Node* head, int key) {
36     int count = 0;
37     while (head != nullptr) {
38         if (head->data == key) {
39             count++;
40         }
41     }
42 }
43
44 }
45 head = head->next;
46 }
47 return count;
48 }
49
50 // Function to count occurrences of 'key' in array
51 int countOccurrencesArray(int arr[], int size, int key) {
52     int count = 0;
53     for (int i = 0; i < size; i++) {
54         if (arr[i] == key) {
55             count++;
56         }
57     }
58     return count;
59 }
60
61 int main() {
62     // Linked List approach
63     Node* head = NULL;
64     int linkedListData[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
65     int linkedListSize = sizeof(linkedListData) / sizeof(linkedListData[0]);
66     for (int i = 0; i < linkedListSize; i++) {
67         insertNode(head, linkedListData[i]);
68     }
69     int countLinkedList = countOccurrencesLinkedList(head, 18);
70     std::cout << "Occurrences of 18 in Linked List: " << countLinkedList << std::endl;
71
72     // Array approach
73     int arr[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
74     int size = sizeof(arr) / sizeof(arr[0]);
75     int countArray = countOccurrencesArray(arr, size, 18);
76     std::cout << "Occurrences of 18 in Array: " << countArray << std::endl;
77     return 0;
78 }

```

Output:

```

Occurrences of 18 in Linked List: 2
Occurrences of 18 in Array: 2

...Program finished with exit code 0
Press ENTER to exit console.

```

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Diagrams for Iterations for Searching:

Iteration 1:

```

+-----+
|  Mid: 5  |
+-----+
| Left: [3, 5] |
| Right: [6, 8, 11, 12, 14, 15, 17, 18] |
+-----+

```

Iteration 2:

```

+-----+
|  Mid: 11  |
+-----+
| Left: [6, 8] |
| Right: [11, 12, 14, 15, 17, 18] |
+-----+

```

Iteration 3:

```

+-----+
|  Mid: 7  |
+-----+
| Left: [6] |
| Right: [8] |
+-----+

```

Iteration 4:

```

+-----+
| Found: 8 |
+-----+

```

Code & Output Console:

```
1 #include <iostream>
2
3 int binarySearch(int arr[], int size, int key) {
4     int low = 0;
5     int high = size - 1;
6
7     while (low <= high) {
8         int mid = low + (high - low) / 2;
9
10        if (arr[mid] == key) {
11            std::cout << "Key found at index " << mid << std::endl;
12            return mid;
13        } else if (arr[mid] < key) {
14            low = mid + 1;
15        } else {
16            high = mid - 1;
17        }
18    }
19
20    std::cout << "Key not found" << std::endl;
21    return -1;
22 }
23
24 int main() {
25     int arr[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
26     int size = sizeof(arr) / sizeof(arr[0]);
27     int key = 8;
28
29     binarySearch(arr, size, key);
30
31     return 0;
32 }
```

```
Key found at index 3
...Program finished with exit code 0
Press ENTER to exit console.
```

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

Code & Output:

```
1 #include <iostream>
2
3 int recursiveBinarySearch(int arr[], int low, int high, int key) {
4     if (low > high) {
5         std::cout << "Key not found" << std::endl;
6         return -1;
7     }
8
9     int mid = low + (high - low) / 2;
10
11    if (arr[mid] == key) {
12        std::cout << "Key found at index " << mid << std::endl;
13        return mid;
14    } else if (arr[mid] < key) {
15        return recursiveBinarySearch(arr, mid + 1, high, key);
16    } else {
17        return recursiveBinarySearch(arr, low, mid - 1, key);
18    }
19 }
20
21 int main() {
22     int arr[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
23     int size = sizeof(arr) / sizeof(arr[0]);
24     int key = 8;
25
26     recursiveBinarySearch(arr, 0, size - 1, key);
27
28     return 0;
29 }
```

```
Key found at index 3
...Program finished with exit code 0
Press ENTER to exit console.
```

8. Conclusion

In this activity, I learned the importance of selecting appropriate search algorithms, such as linear and binary searches, for different data structures and problem types. The activity procedure highlighted the strengths of each algorithm, with binary search being more efficient for sorted data, while linear search proved useful for unsorted data. The supplementary task enhanced my understanding by requiring modifications like counting repeated elements and implementing a recursive binary search. I performed well overall, but there is room for improvement in efficiency and adding more robust error-checking mechanisms into my programs created.

9. Assessment Rubric

Honor Pledge:

"I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own."